一、SQL语言核心特点（选择/填空考点）

核心记住**前3个**，高频考**高度非过程化**

1. 综合统一：集数据定义、查询、操纵、控制于一体
2. 高度非过程化：只需说明“查什么”，无需指定“怎么查”，由DBMS完成执行路径
3. 面向集合的操作方式：操作对象/结果均为集合（表、视图等）
4. 语法简单且组合多样：基础语法简洁，可灵活组合实现复杂需求
5. 简洁易学：关键字少，语法贴近自然语言

二、数据定义（DDL）：模式、基本表、索引

2.1 模式的定义与删除

（1）定义模式

```
CREATE SCHEMA <模式名> AUTHORIZATION <用户名>;
```

- 若**省略模式名**，模式名**隐含为用户名**；
- 示例：`CREATE SCHEMA AUTHORIZATION Wang;`（为用户Wang定义模式Wang）；
- 示例2：`CREATE SCHEMA Hello AUTHORIZATION Ye;`（为用户Ye定义模式Hello）。

（2）删除模式

```
DROP SCHEMA <模式名> <CASCADE | RESTRICT>;
```

- `CASCADE`：**级联删除**，删除模式的同时，删除模式下所有表、视图、索引；
- `RESTRICT`：**限制删除**，若模式下有表/视图/索引，拒绝执行删除操作（模式为空时才可删除）。

2.2 基本表的定义、修改、删除

（1）定义基本表

```
CREATE TABLE <表名>(
    <列名1> <数据类型> [列级完整性约束],
    <列名2> <数据类型> [列级完整性约束],
    ...
    [表级完整性约束]
);
```

① 数据类型（常用）

类型	说明
`CHAR(n)`	定长字符串，长度为n
`VARCHAR(n)`	变长字符串，最大长度n
`INT` / `SMALLINT` / `BIGINT`	整型（不同精度）
`FLOAT` / `DOUBLE`	浮点型/双精度浮点型
`DATE`	日期型，格式YYYY-MM-DD

类型	说明
DATE	日期型，格式 YYYY-MM-DD

② 完整性约束（核心考点）

约束	说明	适用级别
`NOT NULL`	非空，列值不能为NULL	列级
`UNIQUE`	唯一，列值不能重复	列级/表级
`PRIMARY KEY`	主键，隐含`NOT NULL+UNIQUE`	列级（单属性）/表级（多属性）
`FOREIGN KEY`	外键，参照其他表的主键	列级/表级
`CHECK`	自定义条件约束（本课程暂不考）	列级/表级

- **列级vs表级**：仅涉及**单个属性**的约束，可定义在列级/表级；涉及**多个属性**的约束（如复合主键），**必须定义在表级。**

③ 示例：学生-课程-选课表（经典案例）

```sql
-- 学生表Student（单属性主键，列级约束）
CREATE TABLE Student(
    SNO CHAR(9) PRIMARY KEY,  -- 学号，主键（列级）
    SNAME VARCHAR(40) UNIQUE, -- 姓名，唯一
    SSEX CHAR(2),             -- 性别
    SBIRTHDAY DATE,           -- 出生日期
    SDEPT VARCHAR(40)         -- 专业
);

-- 选课表SC（复合主键，表级约束；外键参照）
CREATE TABLE SC(
    SNO CHAR(9),
    CNO CHAR(6),
    GRADE INT,
    PRIMARY KEY(SNO,CNO),     -- 复合主键（表级）
    FOREIGN KEY(SNO) REFERENCES Student(SNO), -- 外键参照学生表
    FOREIGN KEY(CNO) REFERENCES Course(CNO)   -- 外键参照课程表
);
```

（2）修改基本表

```sql
ALTER TABLE <表名>
[ADD <新列名> <数据类型> [完整性约束]]  -- 添加新列
[ALTER COLUMN <列名> <新数据类型>]       -- 修改列的数据类型
[ADD <表级完整性约束>]                   -- 添加表级约束
[DROP <完整性约束名>]                    -- 删除约束
[RENAME COLUMN <旧列名> TO <新列名>];    -- 列重命名
```

- 示例1：为Student表添加邮件列`SMAIL`（变长字符串45）

```sql
ALTER TABLE Student ADD SMAIL VARCHAR(45);
```

- 示例2：将Student表的`SBIRTHDAY`由`DATE`改为`VARCHAR(20)`（DATE占19字节，新类型长度≥19）

```sql
ALTER TABLE Student ALTER COLUMN SBIRTHDAY VARCHAR(20);
```

- 示例3：为Course表添加“课程名非空”约束

```sql
ALTER TABLE Course ADD CONSTRAINT CNAME_NOTNULL CHECK(CNAME IS NOT NULL);
```

（3）删除基本表

```sql
DROP TABLE <表名> <CASCADE | RESTRICT>;
```

- 规则同模式删除：`CASCADE`级联删除，`RESTRICT`限制删除。

2.3 索引的建立、修改、删除（加速查询）

（1）建立索引

```sql
CREATE [UNIQUE] [CLUSTER] INDEX <索引名>
ON <表名>(<列名1> [ASC|DESC], <列名2> [ASC|DESC], ...);
```

- `UNIQUE`：**唯一索引**，索引值与表中记录一一对应，无重复；
- `CLUSTER`：**聚集索引**，表中数据按索引列物理排序；
- `ASC`/`DESC`：升序/降序，**默认升序（ASC）**；
- 可基于**单列/多列**建立索引，多列用逗号分隔。

示例：为学生-课程-选课表建索引

```sql
-- Student表按姓名升序建唯一索引
CREATE UNIQUE INDEX IDX_SNAME ON Student(SNAME ASC);
-- Course表按课程名升序建唯一索引（默认ASC，可省略）
CREATE UNIQUE INDEX IDX_CNAME ON Course(CNAME);
-- SC表按学号升序、课程号降序建唯一索引
CREATE UNIQUE INDEX IDX_SC ON SC(SNO ASC, CNO DESC);
```

（2）修改索引（仅重命名）

```sql
ALTER INDEX <旧索引名> RENAME TO <新索引名>;
```

- 示例：将`IDX_SNAME`改为`ST_NAME`

```sql
ALTER INDEX IDX_SNAME RENAME TO ST_NAME;
```

（3）删除索引

```sql
DROP INDEX <索引名>;
```

2.4 模式与基本表的关联方式

1. 建表时**显式指定模式**：`CREATE TABLE <模式名>.<表名>(...);`
2. 建模式时**同时建表**：`CREATE SCHEMA <模式名> AUTHORIZATION <用户名> CREATE TABLE <表名>(...);`
3. **设置搜索路径**：`SET SEARCH_PATH TO <模式名1>,<模式名2>;`（默认路径含`public`）；
   - 查看搜索路径：`SHOW SEARCH_PATH;`。

三、数据查询（DQL）：核心重点，考法多样（选择/填空/简答/编程）

3.1 通用语法格式

```
SELECT [ALL | DISTINCT] <目标列表达式1>,<目标列表达式2>,...
FROM <表名/视图名> [,<表名/视图名>...]
[WHERE <条件表达式>]
[GROUP BY <列名1> [HAVING <条件表达式>]]
[ORDER BY <列名2> [ASC|DESC]];
```

- 子句执行顺序：`FROM` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `ORDER BY`；
- 后续所有查询均基于此格式扩展，分**单表/连接/嵌套/集合/派生表查询**五类。

3.2 单表查询（仅涉及一张表）

（1）查询指定列/全部列/计算列

- 指定列：`SELECT SNO, SNAME FROM Student;`
- 全部列：用`*`简化，`SELECT * FROM Student;`
- 计算列：基于原有列做运算，可通过`AS`起别名（`AS`可省略）

```
-- 查询学生姓名+年龄，年龄=当前年份-出生年份（Kingbase语法）
SELECT SNAME, EXTRACT(YEAR FROM CURRENT_DATE) - EXTRACT(YEAR FROM SBIRTHDAY) AS 年龄
FROM Student;
```

（2）去除重复行：`DISTINCT`

- 作用：删除查询结果中的重复记录，**默认保留重复（ALL）**；
- 示例：查询选修了课程的学生学号（避免同一学生多次出现）

```
SELECT DISTINCT SNO FROM SC;
```

（3）WHERE子句：条件筛选（核心，多考点）

支持**比较谓词、范围、集合、字符匹配、空值、逻辑运算**，组合使用实现复杂条件。

条件类型	运算符/关键字	示例
比较运算	=, >, <, >=, <=, <>/!=	`GRADE >= 60`（成绩及格）
范围查询	`BETWEEN ... AND ...` / `NOT BETWEEN ... AND ...`	`SAGE BETWEEN 18 AND 22`（年龄18-22）
集合查询	`IN` / `NOT IN`	`SDEPT IN ('CS','IS')`（专业为计科/智能）
字符匹配	`LIKE` / `NOT LIKE`（通配符）	`SNAME LIKE '叶%'`（姓叶）
空值查询	`IS NULL` / `IS NOT NULL`	`GRADE IS NULL`（无成绩）
逻辑运算	`AND`（且），`OR`（或），`NOT`（非）	`SDEPT='CS' AND SAGE <20`（计科且20岁以下）

通配符规则（`LIKE`专用，高频考）

- `%`：匹配**任意长度**的字符串（包括0个字符）；
- `_`：匹配**单个**字符；
- 转义字符：若查询内容含`%`/`_`，用`ESCAPE`指定转义符（如`LIKE 'java\_L%' ESCAPE '\'`，匹配`java_L`开头的字符串）。

（4）ORDER BY子句：排序

- 按**单列/多列**排序，`ASC`升序（默认），`DESC`降序；
- 空值的排序次序由DBMS决定；
- 示例：SC表按课程号升序、同一课程按成绩降序

```
SELECT * FROM SC ORDER BY CNO ASC, GRADE DESC;
```

（5）聚集函数：统计计算（仅作用于数值型列，除COUNT(*)）

函数	说明	示例
`COUNT(*)`	统计表中总元组个数	`COUNT(*)`（选课总记录数）
`COUNT([DISTINCT]列名)`	统计列中非空值个数，DISTINCT去重	`COUNT(DISTINCT SNO)`（选课学生数）
`SUM(列名)`	计算列值总和	`SUM(GRADE)`（成绩总和）
`AVG(列名)`	计算列值平均值	`AVG(GRADE)`（成绩平均值）

`MAX(列名)`	求列值最大值	`MAX(GRADE)`（最高分）
`MIN(列名)`	求列值最小值	`MIN(GRADE)`（最低分）

（6）GROUP BY + HAVING：分组+分组筛选（高频易混考点）

- `GROUP BY`：按指定列**分组**，同列值为一组，聚集函数作用于**每组**；
- `HAVING`：筛选**分组后**的结果，**可与聚集函数连用**；
- ❗ 核心区别：`WHERE`作用于**基本表/视图**，筛选**元组**，**不能与聚集函数连用**；`HAVING`作用于**分组**，筛选**组**，**可与聚集函数连用。**

示例1：求各课程号及选修人数

```
SELECT CNO, COUNT(DISTINCT SNO) AS 选课人数
FROM SC
GROUP BY CNO;
```

示例2：求平均成绩≥90的学生学号和平均成绩（经典易错题）

```
-- 错误：WHERE不能连用聚集函数
-- SELECT SNO, AVG(GRADE) FROM SC WHERE AVG(GRADE)>=90 GROUP BY SNO;
-- 正确：GROUP BY后用HAVING
SELECT SNO, AVG(GRADE) AS 平均成绩
FROM SC
GROUP BY SNO
HAVING AVG(GRADE) >= 90;
```

3.3 连接查询（涉及两张及以上表，核心）

（1）基本概念

- 连接条件/连接谓词：关联多张表的条件，格式`<表名1>.<列名1> <比较符> <表名2>.<列名2>`；
- 连接字段：连接谓词中的列名，通常为**主键-外键**对应关系；
- 等值连接：比较符为`=`的连接；
- 自然连接：**等值连接的特例**，删除目标列中**重复的连接字段。**

（2）等值连接 vs 自然连接（示例）

```
-- 等值连接：Student与SC连接，SNO重复出现
SELECT Student.*, SC.* FROM Student, SC WHERE Student.SNO = SC.SNO;
-- 自然连接：删除重复的SNO，只保留一次
SELECT Student.SNO, SNAME, SSEX, CNO, GRADE FROM Student, SC WHERE
Student.SNO = SC.SNO;
```

（3）复合条件连接

连接谓词 + 选择谓词，用`AND`组合；
示例：查询选修CNO='81001'且成绩≥80的学生姓名+学号

```
SELECT Student.SNO, SNAME FROM Student, SC
WHERE Student.SNO = SC.SNO AND SC.CNO='81001' AND SC.GRADE>=80;
```

（4）自身连接

表与自身连接，需为表起**不同别名**区分，所有列名需加表别名前缀；
示例：查询课程的**间接先修课**（先修课的先修课）

```
SELECT FIRST.CNO, SECOND.CPNO
FROM Course FIRST, Course SECOND
WHERE FIRST.CPNO = SECOND.CNO AND SECOND.CPNO IS NOT NULL;
```

（5）多表连接

≥3张表的连接，依次通过主键-外键关联；
示例：查询学生姓名、选修课程名、成绩（Student+SC+Course）

```
SELECT Student.SNAME, Course.CNAME, SC.GRADE
FROM Student, SC, Course
WHERE Student.SNO = SC.SNO AND SC.CNO = Course.CNO;
```

3.4 嵌套查询（子查询，查询块嵌套）

- 查询块：单个`SELECT-FROM-WHERE`语句；
- 外层查询/父查询：外层的查询块；
- 内层查询/子查询：嵌套在内部的查询块；
- 分类：**不相关子查询**（子查询不依赖父查询）、**相关子查询**（子查询依赖父查询的列值）。

（1）带`IN`的子查询（最常用，不相关子查询）

子查询返回**一个集合**，父查询用`IN`判断是否在集合中；
示例：查询与“哈哈”同专业的学生学号+姓名

```
SELECT SNO, SNAME FROM Student
WHERE SDEPT IN (
    SELECT SDEPT FROM Student WHERE SNAME='哈哈'  -- 子查询：查哈哈的专业
);
```

（2）带比较运算符的子查询

子查询**返回单值**时，可用`=,>,<,>=,<=`等比较运算符；
示例：查询学生“张三”超出自己选修课程平均成绩的课程号

```
SELECT SNO, CNO FROM SC X  -- 父查询表起别名X
WHERE X.GRADE >= (
    SELECT AVG(GRADE) FROM SC Y  -- 子查询表起别名Y
    WHERE Y.SNO = X.SNO  -- 关联父查询，相关子查询
);
```

（3）带`ANY/SOME/ALL`的子查询

子查询返回**多值**，需与比较运算符连用，`ANY=SOME`（任意一个），`ALL`（所有）；

组合	说明
`> ANY`	大于子查询结果中的**某个**值
`> ALL`	大于子查询结果中的**所有**值
`< ANY`	小于子查询结果中的**某个**值
`< ALL`	小于子查询结果中的**所有**值

示例：查询非IS专业中，比IS专业**任意一个**学生年龄小的学生姓名

```
SELECT SNAME, SBIRTHDAY FROM Student
WHERE SBIRTHDAY > ANY (
    SELECT SBIRTHDAY FROM Student WHERE SDEPT='IS'
) AND SDEPT != 'IS';
```

（4）带`EXISTS/NOT EXISTS`的子查询（存在量词，相关子查询）

- `EXISTS`：子查询**非空**（有结果），返回`TRUE`；子查询**为空**，返回`FALSE`；
- `NOT EXISTS`：与`EXISTS`相反，子查询**为空**返回`TRUE`，非空返回`FALSE`；
- 子查询的目标列通常用`*`（只需判断是否有结果，无需具体列值）；

示例1：查询选修了CNO='81001'的学生姓名

```
SELECT SNAME FROM Student
WHERE EXISTS (
    SELECT * FROM SC
    WHERE SC.SNO = Student.SNO AND SC.CNO='81001'
);
```

示例2：查询未选修CNO='81001'的学生姓名

```
SELECT SNAME FROM Student
WHERE NOT EXISTS (
    SELECT * FROM SC
    WHERE SC.SNO = Student.SNO AND SC.CNO='81001'
);
```

3.5 集合查询

对两个查询结果做**集合运算**，要求：两个查询结果**列数相同+对应列数据类型一致**；
常用运算：**并（UNION）**、交（INTERSECT）、差（EXCEPT），**UNION为高频考点。**

示例：查询2020秋学期选修81001**或**81002的学生学号（并运算）

```
SELECT SNO FROM SC WHERE SEMESTER='2020秋' AND CNO='81001'
UNION
SELECT SNO FROM SC WHERE SEMESTER='2020秋' AND CNO='81002';
```

- `UNION`会自动去除重复行，保留重复用`UNION ALL`。

3.6 基于派生表的查询

子查询出现在**FROM子句**中，子查询的结果作为**临时派生表**（需起别名），成为主查询的操作对象；
示例：找出每个学生超出自己选修课程平均成绩的课程号（与3.4.2示例等价，写法不同）

```sql
SELECT SC.SNO, SC.CNO
FROM SC, (
    SELECT SNO, AVG(GRADE) AS AVG_GRADE FROM SC GROUP BY SNO  -- 派生表：学生学号+平均成绩
) AS AVG_SC  -- 派生表起别名AVG_SC
WHERE SC.SNO = AVG_SC.SNO AND SC.GRADE >= AVG_SC.AVG_GRADE;
```